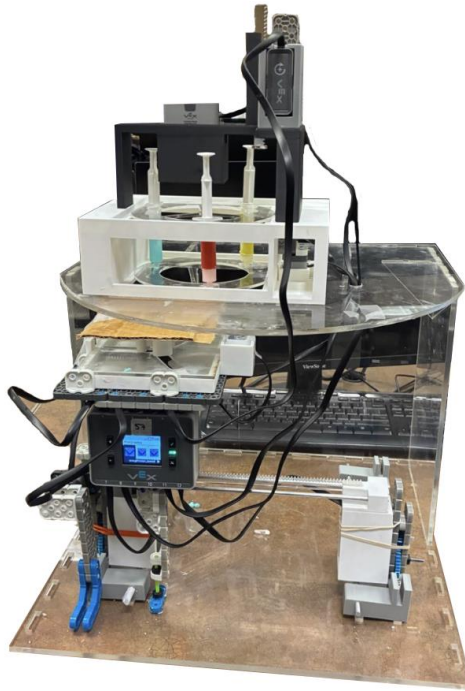




Department of Mechanical and Mechatronics Engineering

ME101 Course Project: Cookie Icing Machine Project Report

A Report Prepared For:
The University of Waterloo

Prepared By:

R.K.S.W., N.G., Trevor Werbin, Ben Szymanski

Department of Mechanical and Mechatronics Engineering
University of Waterloo
200 University Ave W.
Waterloo, Ontario, N2L 3G1

April 6, 2026

Table of Contents

List of Figures.....	ii
Summary.....	iv
1.0 Needs Analysis.....	1
1.1 Background Description.....	1
1.2 Needs Statement.....	1
1.3 Engineering Specification	2
2.0 Conceptual Design	3
2.1 Conceptual Designs.....	3
2.2 Decision Making Matrix	5
3.0 Mechanical Design	5
3.1 Overall Layout.....	5
3.2 Gantry System	6
3.3 Frame and Icing Dispenser System.....	7
3.4 Sensors	8
3.5 Summary of Fabricated Parts	9
3.6 Functions	10
4.0 Verifications of Design.....	11
4.1 Specification Verification	12
4.2 Overall Assessment.....	13
5.0 Software Design & Implementation	14
5.1 Overall Software Operation	14
5.2 Task List for Software Demonstration	15
5.3 Program Breakdown.....	16
5.4 Programming Functions Table	17
5.5 Data Storage	19
5.6 Design Decisions & Tradeoffs	20
5.7 Testing.....	21
5.8 Debugging & Final Program	22
6.0 Project Management.....	22
6.1 Work Distribution	22
6.2 Project Schedules	22

7.0 Conclusion	24
7.1 Design Objectives and Features	24
7.2 Review of Requirements	24
8.0 Recommendations	25
8.1 Mechanical Changes	25
8.2 Software Changes.....	25
References	26
Appendix A – Full Program Code.....	27

List of Figures

Figure 1: Cookie Icing Slicer Tool	1
Figure 2: Conceptual Design Illustration 1	4
Figure 3: Conceptual Design Illustration 2	4
Figure 4: Conceptual Design Illustration 3	4
Figure 5: Annotated CAD Photo of Overall Layout.....	6
Figure 6: Annotated CAD Photo of Gantry System.....	7
Figure 7: Annotated CAD Photo of Icing Dispenser.....	8
Figure 8: Programming Flowcharts 1.....	14
Figure 9: Programming Flowcharts 2.....	14
Figure 10: Work Breakdown Structure.....	22
Figure 11: Initial Gantt Chart Schedule.....	23
Figure 12; Finalized Gantt Chart Schedule.....	23

List of Tables

Table 1: Final Engineering Design Specification.	2
Table 2: Decision Making Matrix.....	5
Table 3: Sensors Utilized in Machine.....	9
Table 4: 3D Printed Parts.....	9

Table 5: Laser Cut Parts.....	10
Table 6: Line by Line Verification of Engineering Specification Design.....	12
Table 7: List of Programming Functions.....	17

Summary

This report presents the design and verification of specifications of an automated cookie icing machine built using the VEX IQ platform for ME101 course. Many cookie icing machines are built for commercial purposes and used for large-scale production, making them too large and expensive for household use. As a result, home cooks lack access to affordable and compact tools to consistently and efficiently ice their cookies. The icing machine allows users to decorate cookies with consistent, repeatable icing patterns via a two-axis gantry system. The device addresses the challenge of achieving consistent and high-quality cookie decoration while increasing accessibility in the kitchen.

The machine was designed to dispense icing in a pattern across a cookie surface using VEX IQ motors to drive the X and Y axes and motors to control the color selection barrel and syringes depositing icing. The designs were created by users through a front-end HTML/JavaScript slicer and then exported into the code.

The machine accurately applied an icing design to a cookie surface while detecting the boundaries of the cookie within a cycle time of under 4 minutes. The major components included a gantry, an icing dispensing mechanism using syringes and a color sensing system to accurately choose the correct colours. In the final design, the cookie tray and brain were attached to the gantry while the icing dispensing apparatus remained stationary.

The software was implemented in VEX C++ and structured around a coordinate array exported from the slicer and executed sequentially by the VEX IQ brain.

Each system has been tested against the engineering specifications, and the machine has been demonstrated to successfully ice a pattern onto a cookie with repeatable accuracy. However, icing consistency remains a limitation due to variations in aspects such as syringe pressure and the presence of air gaps across different portions in the pattern resulting in uneven icing deposition.

1.0 Needs Analysis

1.1 Background Description

Cookie decorating is a popular activity in home baking yet achieving consistent and precise icing patterns by hand requires considerable skill and practice. For individuals with limited mobility or dexterity, manual icing presents a significant challenge, often excluding them from participating in a fun and creative kitchen activity. Many commercial cookie icing machines are designed for large-scale industrial production, making them too large, complex, and costly for personal use. As a result, the average hobbyist lacks access to affordable and compact tools to consistently and efficiently decorate cookies. A small-scale automatic icing machine would make decorating easier and neater while improving accessibility, enabling people of all ages and abilities to participate in icing cookies and have fun in the kitchen.

This project was chosen to address that gap by utilizing VEX IQ components to construct a compact and fully automated cookie icing machine. The workflow begins with the user drawing a desired icing pattern in a custom web-based slicer tool, as seen in Figure 1, dividing the design and exporting coordinate arrays encoded with both the path and the icing colour. These arrays are then manually imported into the VEX C++ program and loaded onto the machine. Once the user places a cookie on the platform, the machine autonomously executes the pre-programmed pattern, initiating gantry movement and the dispensing of icing.

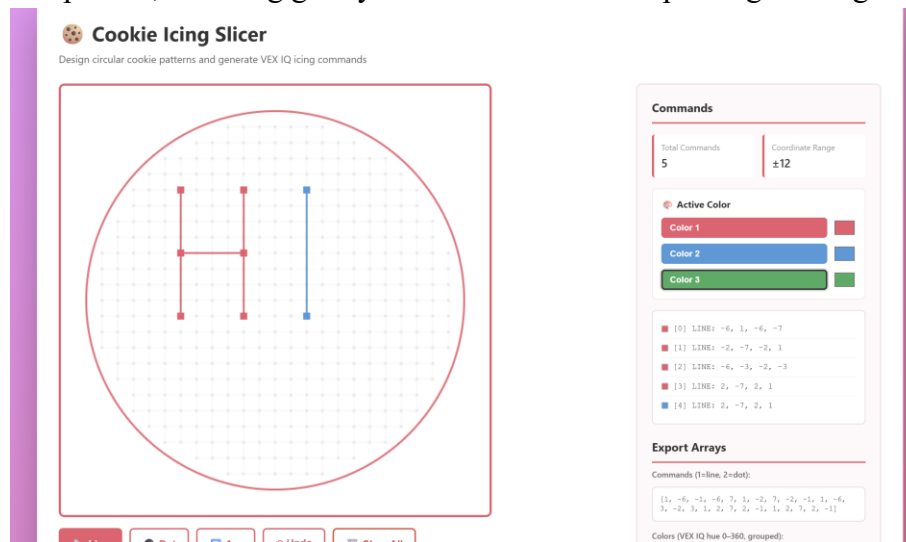


Figure 1: Cookie Icing Slicer tool showing the “HI” pattern programmed for the demonstration

1.2 Needs Statement

A need exists for a system that automatically and accurately applies icing to cookies while preserving the cookie's original orientation and completing the process in under 240 seconds.

1.3 Engineering Specification

Table 1 shows the finalized engineering design specifications used during testing.

Table 1: Final Engineering Design Specification

No.	Characteristic	Relation	Value	Units	Verification Method	Comments
1	Cookie decoration cycle time	<	240	Seconds	Test	Time how long it takes to print a design from start to finish (input cookie and tap sensor to start to cookie being returned to the zero position)
2	Draw lines and curves	N/A	N/A	N/A	Demonstration	Can draw lines and curves using icing and form them into patterns
3	Adaptability of machine to the cookie	<=	4	Inches	Test	Perform test using various sizes of cookies under 4 inches to gather adaptability data.
4	Time to learn and understand operation instructions	<	5	Minutes	Test	Time how long it takes operator to explain the instructions and for the user to start the machine.
5	Print Area	=	64	Inches squared	Test	Since the print area is a square, the edges can be measured using a rules and the area can be calculated.
6	End at zero position (where started)	N/A	N/A	N/A	Demonstration	The robot always returns the cookie plate to the zero

						position for every time it operates.
7	Cookie plate stability	N/A	N/A	N/A	Demonstration	As the gantry moves, the cookie plate will be stable and upright without any rocking movement/torsion.
8	Icing Colour Dispensing	N/A	N/A	N/A	Demonstration	Machine can apply multiple colours of icing on a single design.
9	Number of Icing Colours	=	3	N/A	Examination	Visually identify that there are three syringes containing three different icing colours.
10	Cost	<	300	\$	Analysis	Calculate total cost using receipts from RPC and other places/stores.

2.0 Conceptual Design

The cookie icing machine was developed through a structured research and ideation process, beginning with the identification of the machine's four sub-functions: pattern input, cookie positioning, icing output, and icing colour selection.

2.1 Conceptual Designs

Figure 2 shows the first concept that was developed using morphological analysis and AI-assisted research [1]. This design features a dual-axis (XY) bottom gantry attached to and moving in tandem with the cookie plate. In the top half of the design, a motor-driven cam periodically pushes a syringe plunger, in a motorized turret, to dispense icing. This design was used as the 'datum' for the design matrix in Figure 4.

Figure 3 depicts the second concept which was developed using morphological analysis and video inspiration. This design also uses an XY gantry but replaces the cam with a funnel-based dispensing mechanism that uses a motor-actuated press plate to squeeze the icing downward. Colour selection is achieved by having the gantry pick up different syringes from a holder. While

the passive funnel reduces actuated components, the viscosity of the icing would likely result in a blockage and dispensing imprecision.

The third concept, as shown in Figure 4, was generated through brainstorming and morphological analysis. This design features a rotating platform carrying multiple syringes, with icing directed downward through a funnel. Though creative, the rotating platform significantly increases mechanical complexity and spatial footprint, making it the least practical of concepts.

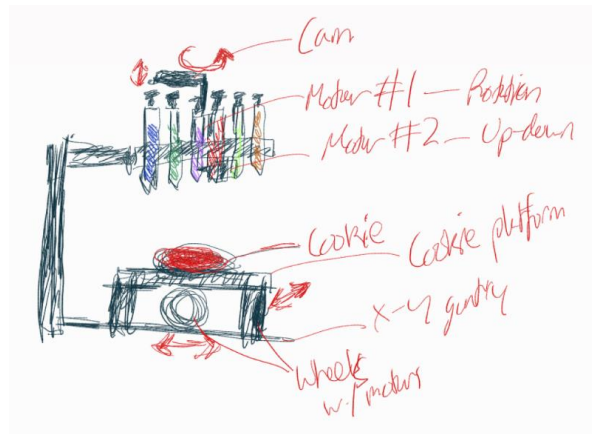


Figure 2: Conceptual Design Illustration 1

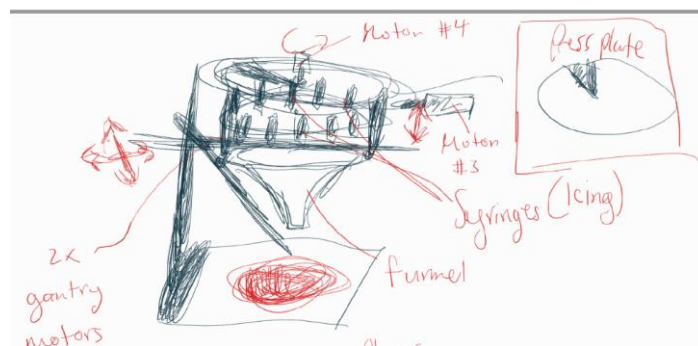


Figure 3: Conceptual Design Illustration 2

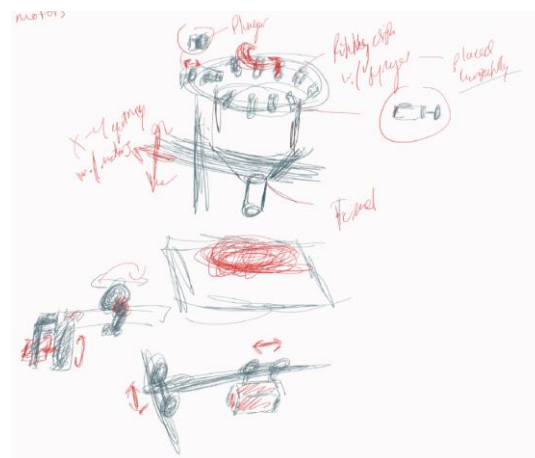


Figure 4: Conceptual Design Illustration 3

2.2 Decision Making Matrix

Table 2 is the decision-making matrix used within the selection process. The final concept selected was concept 1.

Table 2: Decision Making Matrix

Criteria	Concept 1 (Datum)	Concept 2	Concept 3
Speed (time for completion) (performance)	0	-1	-1
Adaptability (accommodation of multiple cookie sizes)	0	0	-1
Icing application (efficiency and accuracy) (performance)	0	1	0
Cost	0	-1	-1
Safety	0	1	1
Total	0	0	-2

3.0 Mechanical Design

3.1 Overall Layout

The cookie icing machine, as depicted below in Figure 5, involves a two-axis cartesian gantry system built from a mixture of VEX IQ components alongside 3D printed and laser cut acrylic structural components. This machine consists of two main subsystems stacked vertically. The first is the lower gantry assembly, composed of two separate rack and pinion systems for X and Y axis motion, permitting planar translation of the cookie, alongside the cookie plate itself, which is mounted at the top of the rack and pinion systems. The second is the upper icing system which is comprised of a rotating barrel of three syringes and a motorized plunger mechanism for icing dispensing. The VEX brain is mounted under the cookie plate to maximize proximity to all motor and sensor components and reduce the total length of wiring needed. The two subsystems are connected to each other by a laser cut acrylic enclosure, which envelops the entire build

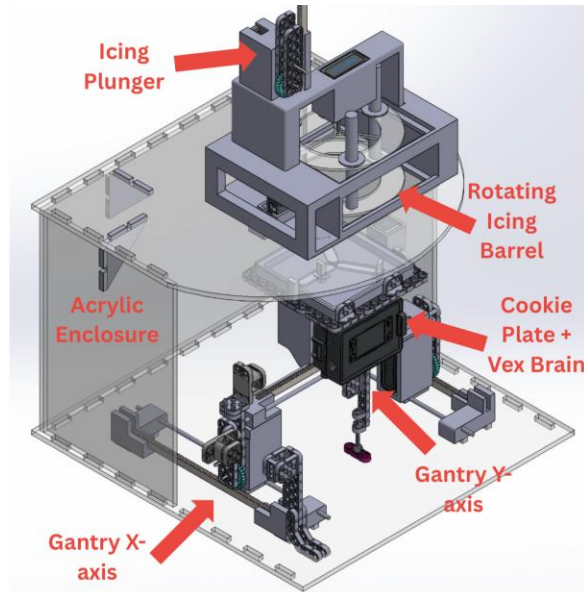


Figure 5: Annotated CAD Photo of Overall Layout

3.2 Gantry System

The gantry was designed to allow 8 inches of travel along each axis independently. A rack and pinion system was selected as the most straightforward mechanism to meet this requirement, and the components were standardized between the X and Y axes wherever possible to simplify manufacturing and improve modularity.

The X-axis forms the base of the gantry and uses two motors paired with two rack and pinion assemblies, one on each side, to distribute the load of the Y-axis assembly evenly. The Y-axis sits on top of the X-axis and is driven by a single motor.

Both racks were designed to mate with the VEX IQ 36T gear and were laser-cut from acrylic for improved dimensional precision. Each rack is 9 inches long, one inch beyond the 8-inch travel range, to provide clearance for the pinion at the endpoints of travel.

The racks are supported at each end by stoppers, which also serve as mounting points for aluminum guide rods. These rods act as rails to constrain and stabilize the pinion carriage during motion. The X-axis uses one aluminum rod per side, while the Y-axis uses two, to better support the additional load of the cookie plate and VEX Brain. The X-axis stoppers are anchored to the laser-cut acrylic baseplate, while the Y-axis stoppers mount directly to the X-axis pinion carriage.

The pinion assemblies were standardized across both sides of the X-axis, though modifications were required for the Y-axis to accommodate the additional load of the Brain. Each pinion system used a 12T to 36T gear reduction gearbox built entirely from VEX components. 3D printed casings were designed to enclose the motors and gearboxes, serving a dual purpose as structural attachment points. On the X-axis, these casings acted as mounting points for the Y-axis stoppers which held up the Y-axis. On the Y-axis, the casing served as the

mounting point for both the VEX Brain and the cookie plate. In addition, a stabilizing rod was later added to the Y-axis casing, to solve wobbling issues due to the height of the gantry.

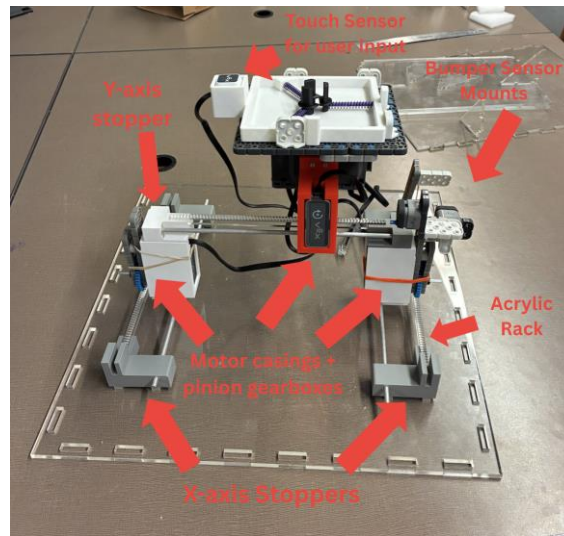


Figure 6: Annotated CAD Photo of Gantry System

3.3 Frame and Icing Dispenser System

As seen in Figure 7, the icing module attaches to the laser-cut acrylic enclosure via the roof. In order to achieve the functions of being able to use multiple icing colors, the module was split further into two sub-systems, the injector and the barrel. The barrel is powered by one motor, and using two sandwiched acrylic plates holds three syringes containing different icing colors arranged radially at 120 degree intervals. The rotation of the barrel permits selection of the icing color, and the barrel is rotated such that the tip of the selected syringe lines up to the same position each time. The injector system consists of a plunger which is then used to compress the syringe, as to inject the icing onto the cookie's surface.

The injector was mounted in a manner that provided slightly less than an inch of tolerance between the cookie plate and the nozzle, as to accommodate cookies of all sizes and prevent icing smearing against the roof of the enclosure during printing. The barrel and injector were enclosed within a larger 3D-printed structure that held all motors and sensors together. The acrylic roof had various slots and cutouts, which permitted the barrel and wire to poke through, as well as allow for extrusions on the underside of the icing dispenser structure to slot in and attach.

The icing module, as a simpler system, was essentially combined with the frame throughout the build and testing process. The frame consisted of four acrylic plates, two for the sides, one for the back, and the roof, which slotted into the baseplate containing the gantry by means of interlocking slots and extrusions. Gussets were added between the side, back and roof

plates, to provide additional structural stability and hold up the cantilevered weight of the icing module better.

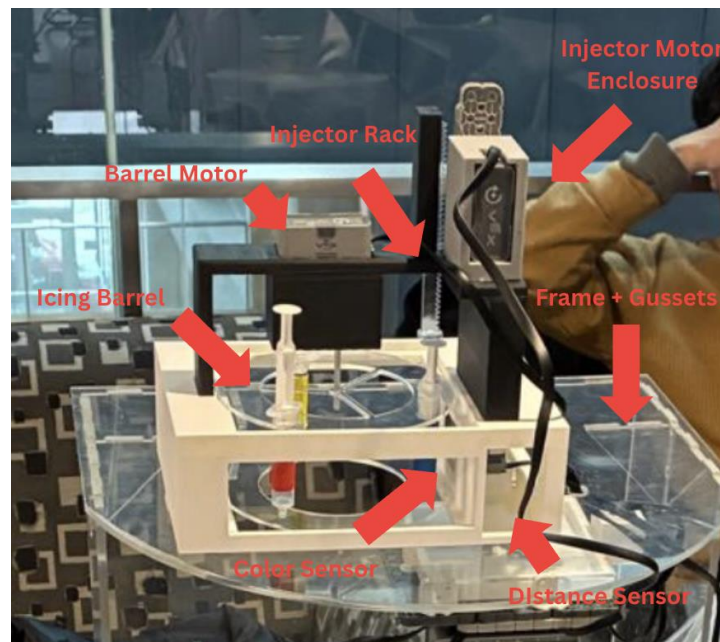


Figure 7: Annotated Photo of Icing Dispenser System

3.4 Sensors

The cookie icing machine relies on two primary sensors for autonomous operation: a bumper sensor to initiate the icing sequence, and an optical color sensor to detect and select the appropriate icing color during operation. These sensors were characterized experimentally in Mech Design Assignment 3, and the findings were used to inform the final design.

The bumper sensor was chosen to be the zeroing device, due to its low rate of false readings for time-dependent tasks greater than one second, such as in this case zeroing the gantry. Timer accuracy testing revealed that the VEX IQ bumper has incredibly low percentage errors for readings between 2 and 8 seconds, which however sharply to a error greater than 80% for very short quick tap inputs. This indicates that the bumper would perform excellently for the relatively non-transient zeroing function.

The optical color sensor was evaluated for its ability to distinguish between multiple icing colors. Testing demonstrated that the sensor identified all six predefined color hues, such as red, yellow, green, cyan, blue, and purple, when the LED was on and the sensor was held at an appropriate distance from the target surface. The effective operating range was found to be between 0 and approximately 32 to 67 mm depending on the color, beyond which hue readings became unreliable.

The cookie icing machine has a set of various sensors, which are illustrated in the table below, as well as the diagram. Table 3 below summarizes the sensors used in the design.

Table 3: Sensors Utilized in Machine

Sensor	Location	Function
Bumper Switch(X-axis)	Left end of X-axis rail	Reference point to detects home position for X and Y axis during zeroing
Bumper Switch(Y-axis)	Left end of Y-axis rail	
Optical Sensor	Facing Icing Barrel	Reads hue at each 120 degree stop to identify available icing colours during rotation
TouchLED	Next to Cookie Plate	Serves as user start button to initiate the icing cycle
Distance Sensor	On top of roof, facing cookie plate	Used to detect cookie size by running the sensor over it to detect where the cookie is.

3.5 Summary of Fabricated Parts

As seen in Table 4 below, 3D printing was used to make parts with dimensional accuracy. PLA of 5% adaptive cubic infill was chosen for low cost and acceptable strength.

Table 4: 3D Printed Parts

Part Name	Quantity	Location	Function
X-axis Motor Casing	2x	Around x-axis motors	Integrating the VEX IQ gearbox with fabricated parts
Y-axis Motor Casing	1x	Around y-axis motors, behind brain	
X-axis stopper	4x	On top of baseplate	Holding the aluminum rod guide rails and racks
Y-axis stopper	1x	On top of X-axis motor casing	
Y-axis Casing add-on	1x	Sandwiching Y-axis rack with Y-axis casing	Attaching the brain and cookie plate to the second aluminum y-axis guide rail
Icing Structure (two halves)	2x	On top of roof	Holding the injector and barrel together
Injector motor casing	1x	On top of icing structure	Holding injector motor
Barrel motor casing	1x		Holding barrel motor
Cookie Plate Case	1x	On top of VEX brain and y-axis motor casing	Keeping cookie mounted to the cookie plate
Cookie Plate Jaws	3x		

Laser cutting acrylic was used for parts that were only used on one plane as seen in Table 5.

Table 5: Laser Cut Parts

Part Name	Quantity	Function
Baseplate	1x	Keeping gantry and remainder of frame stabilized
Sideplate	2x	Providing stability of icing module, and aesthetic appeal of the overall design
Backplate	1x	
Roof	1x	Keeping the icing module stabilized
Gussets	4x	Increasing the structural integrity of the frame
Racks	3x	Creating linear motion in the gantry.

3.6 Functions

The cookie icing machine must perform four primary functions: transporting the cookie to the correct position, selecting the desired icing color, dispensing icing onto the cookie surface, and detecting the cookie boundary.

Cookie transport is achieved by the dual-axis rack and pinion gantry, which moves the cookie plate in the X and Y directions independently. Color selection is performed by the motorized barrel, which rotates in 120° increments to align the desired syringe with the injector. Icing dispensing is carried out by the injector's motor-driven plunger, which compresses the selected syringe to extrude icing through the nozzle. Finally, cookie boundary detection is handled by the distance sensor mounted on the roof, which scans over the cookie plate to determine the cookie's size and position before the icing cycle begins.

These primary functions can be broken down into nine individual functions seen below.

1. Line Drawing (drawLine) - Ryan

The machine travels to the first coordinate specified in the path array, then drives to the second coordinate while dispensing icing at a constant rate, tracing a continuous line between the two points. Testing will use two points at a known distance to verify gantry positioning accuracy.

2. Dot Drawing (drawPoint) - Ryan

Identical to the line draw in terms of positioning, but the gantry remains stationary at the target coordinate and dispenses icing for a fixed duration to form a dot. Testing will evaluate icing viscosity and spread, similar in concept to a hybrid between concrete slump test and filament extrude test.

3. Zeroing/Calibration (zeroGantry) - Ryan

Both X and Y motors drive in one direction until their respective bumper switches are triggered, at which point the motor encoders are reset to zero. Testing will compare encoder-reported distance to physically measured travel distance.

4. Injection (syringeStart/syringeStop/syringeDot) - Trevor

The plunger motor rotates the rack downward at a set rate to dispense icing, with the encoder position saved after each actuation so subsequent dispenses can continue from the correct position. The optimal flow rate will be determined empirically, integrated as part of the dot drawing function testing.

5. Rotate to color (rotateBarrelTo) - Brendan

The icing barrel rotates using motor encoder feedback to approximate the angle turned and align the correct syringe. All paths of a given colour are completed before the barrel rotates to the next, minimizing unnecessary switching. Testing will verify that the selected syringe is centered under the plunger rack.

6. Color check (indexBarrelColors) - Brendan

The barrel rotates through its full index, and the colour sensor reads and maps each loaded syringe to the colours specified in the frontend design. Testing will analyze sensor hue values to confirm accurate colour identification.

7. Execute Plan (executePlan) - Ryan

The main execution loop for the icing run. Iterates through the directions and color arrays, calling the commands of drawLine() or drawPoint() depending on the instruction type. Increments a counter after each draw operation, which triggers a zeroGantry() re-calibration every 5 draws to prevent encoder drift from accumulating over a long run. Testing will involve running a full multi-colour design and verifying that the output matches the frontend pattern.

8. Resize Grid (secToMainX / secToMainY) - Nathan

The calculated size of the cookie will be used to resize the coordinate system for the icing pattern to maintain accuracy. Testing will involve measurement of sizes using a square pattern of known size that is on the perimeter of the cookie.

9. Size of Cookie (calibrateCookie) - Nathan

A calibrateCookie function was designed to detect cookie dimensions using three spring-loaded prongs on the cookie plate, with the calculated size passed to other functions at runtime. The function was not integrated into the final program due to insufficient time for testing and validation. Instead, the user is prompted to manually input the cookie's dimensions via the VEX Brain controller at the start of each run

4.0 Verifications of Design

4.1 Specification Verification

Table 1 shows the final engineering design specification that Table 6 references for verification purposes. In addition, Table 6 mentions whether the engineering specification passed or failed the testing done.

Table 6: Line by Line Verification of Engineering Specification Design

Characteristic	Pass or Fail	Test Performed	Comments
Cookie Decoration Cycle Time	Pass	20 cycles of the machine were timed using a stopwatch.	Less complicated designs had an average start to finish time of around 110 seconds, while more complicated designs saw times around 170 seconds.
Draw Lines and Curves	Pass	A variety of designs and patterns were tested including squares, circles, arcs, words, etc.	The machine was able to accurately apply icing in the desired pattern. However, due to some software complications, the patterns were inverted.
Adaptability of the Machine to the Cookie	Fail	Store bought cookies were manipulated to test different sizes of cookies ranging from 1-4 inches in diameter.	While the adaptability itself was good and satisfied the specification requirement, the force applied from the springs in the plate design caused the cookies to crumble and led to cardboard being used for testing and the demonstration instead.
Time to Learn Instructions	Pass	New users during demonstration day were taught how to use the machine and were timed.	From a sample size of 12 individuals, the average time taken from the first click of their design to the start button being pressed was 2 minutes 42 seconds.
Print Area	Pass	The total distance the gantries moved in the X and Y were measured with a ruler.	While the racks themselves are longer than the required 8 inches, the printable length in the X was measured to be 8 inches, and in the Y, it was measured to be 8.05 inches. This gives a printable area of 64.4 square inches.

Zero Position	Pass	The cookie plate was started in a different position for 25 demonstrations.	In every demonstration, both X and Y gantries zeroed out to start operation.
Cookie Plate Stability	Pass	The stability was tested using 6 different weight cookies, each tested 3 times.	The maximum angle of depression measured from the tests (measuring from front to back) was approximately 3 degrees. This provides sufficient stability for the cookie, which was shown in demonstrations.
Icing Colour Dispensing/ Number of Colours	Pass	Many different colours were used on the front end slicer to see how the machine would react.	When loaded with red, yellow, and blue icing (visually confirmed), the machine functions as normal and dispenses the icing colour that is closest to the hue the user selected, which was shown in demonstration. Sometimes not every colour is used depending on the hue the user chooses.
Cost	Pass	All transaction data was stored in a spreadsheet as the project progressed.	The total cost after adding up all transactions is \$237.25. This is below the \$300 maximum in the design specification.

4.2 Overall Assessment

Overall, the cookie icing machine worked as intended. Nine of ten engineering specifications were met, with the only failure being cookie adaptability. The spring-loaded plate caused cookies to crumble during testing, so cardboard substitutes were used for the demonstration instead. Additionally, patterns were discovered to be inverted due to a software issue, though this did not affect the machine's ability to execute the programmed design accurately. The verification results in Table 6 are supported by testing conducted throughout the development process. Sensor characterization for the bumper and optical color sensors was performed in Mech Design Assignment 3, establishing operating ranges that informed their placement in the final design. Software testing was conducted iteratively as described in Sections 5.7 and 5.8, with each function validated individually before full system integration, resolving issues including icing drip, air bubble purging, and encoder drift. Gantry accuracy was verified by comparing encoder-reported positions against physically measured travel distances, with periodic re-zeroing introduced to address cumulative error. The core function of the machine,

autonomously applying a user-defined multi-colour icing pattern within the time constraint, was demonstrated successfully and repeatably. Thus, the overall assessment is positive in declaring success of the cookie icing machine in reaching most design goals specifications.

5.0 Software Design & Implementation

5.1 Overall Software Operation

Figures 8 and 9 show the flowcharts created for the software.

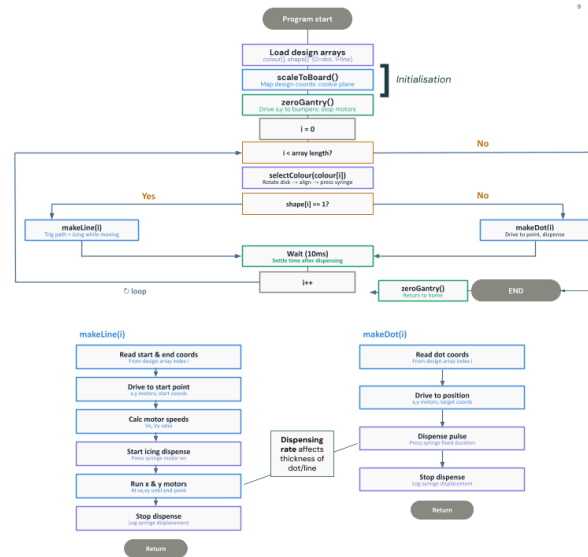


Figure 8: Programming Flowcharts 1

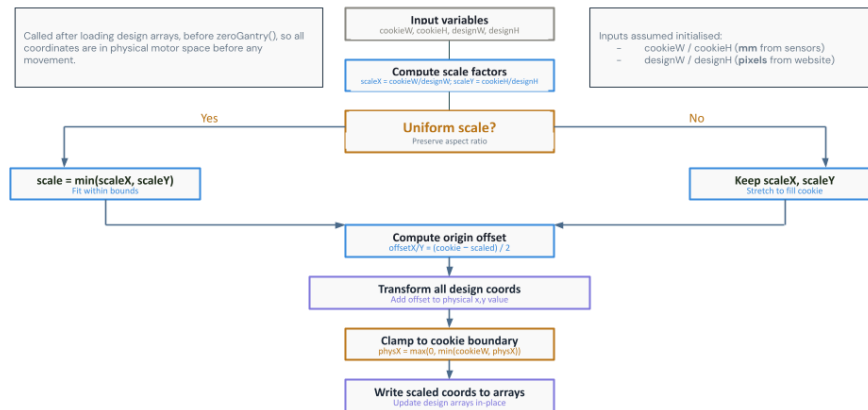


Figure 9: Programming Flowchart 2

Below are some of the user stories for the machine:

User Story 1

Start-Up: As a robot, I want to calibrate my gantry movement so that the icing pattern is

accurate.

Regular Operation: As a robot, I want to have the icing dispensing speed match the gantry movement at a constant ratio, along with reading the icing array number and colour, so that user satisfaction rates stay high.

User Story 2

Shutdown: As a robot, I want to return to the 0 spot when the pattern is finished so that the user knows the print is done or as a robot, I want to shutdown the system when the user presses the TouchLED so that the user can perform their needed actions.

5.2 Task List for Software Demonstration

High Level Software Operation

☐ Website converts design to two arrays of movement/line/dot codes, and colour ID's.

Start Up

- ☐ VEX screen indicates "press to start"
- ☐ Zeros gantry
- ☐ Aligns center of cookie plate to center of syringe.
- ☐ Index barrel colours

Regular Tasks

- ☐ Gantry traces correct shape
- ☐ Colours rotated to the correct active colour, and colour ID's for each function matched to the correct colour on the barrel
- ☐ Zeroes out for every 5 line/dot functions performed.
- ☐ Dispensing/squeeze mechanism works

Communicates w/ User

- ☐ Displays icing hues on screen during indexing
- ☐ Words "Zeroing" displayed on screen during zeroing
- ☐ Words on brain screen prompts user to start program and notifies when its done
- ☐ Displays icing colour match failure if a user-selected colour ID doesn't fall within the tolerance of the barrel icing colours

Shut Down Procedure

- ☐ Gantry zeroes out and hums melody
- ☐ VEX screen displays "done"

Reacts to Environment

- ☐ Incorrect colour input checker (hue tolerance) assigns a colour, or otherwise defaults to the previous colour used and continues from there.
- ☐ Scales coordinates of secondary-cookie based grid to fit the main grid involving the gantry's actual physical print area

☐ Truncates coordinates if beyond main grid, as to keep the design within physical bounds.

Reliability

- ☐ Can replicate results at least twice
- ☐ Doesn't break down during

5.3 Program Breakdown

The program is organized into six functional modules executed sequentially from main().

5.3.1 Startup and User Prompt

On power-on, the Brain plays a startup melody and displays “Press start...” on the screen. The program waits for the TouchLED to be pressed before proceeding. On the side, array sizes for directions[] and colors[] are computed.

5.3.2 Zeroing

The zeroGantry() function involved all three gantry motors (motorXA, motorXB, motorY) spin in reverse at HOME_SPEED simultaneously. The program polls bumperX and bumperY independently, and when each bumper is pressed, the corresponding motors brake and a flag is set. Once both axes are zeroed, all encoder positions are reset to zero and currentX and currentY are set to 0.0. A periodic re-zeroing mechanism (incrementAndCheckCounter()) also calls zeroGantry() after every five strokes during the to account for encoder drift.

5.3.3 Start Point Navigation

The gantry moves to the secondary grid origin via moveToSec(0, 0), which maps to MAIN_CENTER_X and MAIN_CENTER_Y to the center of the cookie surface. All moves use spinToPosition() to ensure that all three motors can run simultaneously, with a loop waiting for all motors to finish spinning before the program continues.

5.3.4 Barrel Colour Calibration

The indexBarrelColors() function sets the optical sensor LED to full power and spins the barrel motor forward at 5% through a full rotation. At each step, the sensor's hue is read and checked against predefined hue windows for each of the three colour slots. When a match is found, the program waits 200 ms for the reading to settle, re-reads the hue, and records the barrel encoder position in barrelColorPos[] and the confirmed hue in barrelColorHue[]. After the scan, zeroBarrel() returns the barrel to encoder zero. If any slot was not detected, a warning is displayed on the Brain screen, and the first valid detected colour is selected as a substitute.

5.3.5 Pattern Execution

The executePlan() function iterates through the directions[] array sequentially. Each entry begins with a command integer: 1 signals a line and 2 signals a dot. Before each call, selectColor() is called with the corresponding entry from colors[], which retracts the syringe,

rotates the barrel to the correct slot, and restores the saved plunger height for that colour. drawLine() moves the gantry to the start coordinate, preloads the syringe, starts continuous dispensing during the gantry movement to the endpoint, then stops and holds the plunger. DrawPoint() moves to the target coordinate and dispenses a fixed-duration dot via syringeDot().

5.3.6 Shutdown

After the pattern completes, the gantry returns to the secondary grid origin and re-zeros via zeroGantry(). finalRetract() then rewinds the active syringe motor to encoder position 0. The Brain displays "Done!" and plays the startup melody once more to signal completion.

5.4 Programming Functions Table

Table 7 depicts the list of programming functions used in the project.

Table 7: List of Programming Functions

Function	Parameters	Description
zeroGantry()	N/A	Drives all axes in reverse until bumpers trigger; resets all encoders to zero
moveToMain(x, y, speed)	target inches, speed %	Moves gantry to absolute main-grid coordinates simultaneously on all three motors
moveXOnly(x, speed)	target inches, speed %	Moves X axis only (motorXA + motorXB) to target position
moveYOnly(y, speed)	target inches, speed %	Moves Y axis only (motorY) to target position
moveToSec(sx, sy, speed)	secondary grid coords	Converts secondary grid coords to main grid via secToMainX/Y(), then calls moveToMain()
secToMainX(sx) / secToMainY(sy)	secondary coord	Scales secondary grid position by (cookieDiameter/2) / SEC_GRID_RADIUS, offsets by center
clampMain(v)	inches	Clamps value to [GANTRY_MIN, GANTRY_MAX] (0.0–8.0 in)
inchesToDegX(in) / inchesToDegY(in)	inches	Converts inches to motor encoder degrees using COUNTS_PER_INCH and axis tune factors

indexBarrelColors()	N/A	Sweeps barrel 360°, reads optical sensor hue, records encoder position per colour slot into barrelColorPos[]
findColorSlot(hue)	requested hue int	Searches barrelColorHue[] for closest match within HUE_TOLERANCE; returns slot index or -1
selectColor(hue, heights)	hue int, plunger heights array	Calls findColorSlot(), saves current plunger position, retracts syringe, rotates barrel, restores plunger height for new slot
rotateBarrelTo(deg)	target encoder degrees	Spins barrel motor to target position at BARREL_MOVE_SPEED
zeroBarrel()	N/A	Returns barrel encoder to 0 by spinning forward or reverse depending on current position
syringePreload(angle)	degrees	Advances syringe forward a fixed amount before a draw to prime icing at nozzle
syringeStart()	N/A	Begins continuous forward syringe spin at SYRINGE_SPEED_LINE
syringeStop()	N/A	Holds syringe, records dispensed amount, updates plungerHeight[] and syringeDist[]
syringeDot()	N/A	Spins syringe forward at SYRINGE_SPEED_DOT for SYRINGE_DOT_MS ms, then holds; updates tracking variables
syringeCheck(slot)	slot index	Displays warning on Brain screen if syringeDist[slot] reaches zero
finalRetract(heights)	plunger heights array	Rewinds active syringe to encoder position 0 at the end of the run
drawLine(x1,y1,x2,y2,heights)	secondary coords, heights array	Moves to start point, preloads and starts syringe, moves gantry to end point while dispensing, stops syringe

<code>drawPoint(x1,y1,heights)</code>	secondary coords, heights array	Moves to coordinate, preloads, calls <code>syringeDot()</code>
<code>executePlan(dirs, dSize, colors, cSize, heights)</code>	direction/color arrays + sizes + heights	Main execution loop; parses command integers from <code>DIRECTIONS[]</code> , dispatches to <code>drawLine()</code> or <code>drawPoint()</code>
<code>incrementAndCheckCounter()</code>	N/A	Increments <code>functionCounter</code> ; triggers <code>zeroGantry()</code> every <code>ZERO_INTERVAL</code> (5) draw calls
<code>calibrateCookie()</code>	N/A	Detects three cookie rim prongs via distance sensor, computes circumcenter geometry, updates <code>cookieDiameter</code> , <code>MAIN_CENTER_X/Y</code>

5.5 Data Storage

The frontend slicer for the user's design outputs two flat arrays that the user pastes directly onto the source code before each run, namely the directions array and the colors array.

The directions array is a flat double array which encodes all draw commands sequentially. Inspiration was taken from CNC-style G-Code to create this system. Each command begins by having the program read in a type flag from the array, 1 for a line, and 2 for a dot or point. Based on this type flag, the program then reads in 4 additional values or 2 additional values for a line(`x1, y1, x2, y2`) and dot(`x1, y1`) respectively.

All coordinates are given by the slicer with respect to the cookie, and these coordinates need to be translated from the reference frame of the cookie to the gantry. As such, there are two coordinate grids, the main and secondary grid, which handle this. The secondary grid system is a unitless coordinate space with a maximum coordinate radius of 12 relative to (0,0), the center of the cookie. This secondary grid then gets scaled to physical inches at runtime based on the given cookie size, and this thus allows for the secondary grid coordinates to be translated into the main grid and vice versa for gantry movement.

The colors array is an integer array parallel to the directions array, where each entry is a hue value from 0 to 360, corresponding to the color used for the given parallel draw command. In essence, the n-th entry in the colors array will map to the n-th type flag or command parsed from the directions array.

In addition to these two flat arrays, the user has access to `cookieDiameter`, which is a double variable representing the cookie size. This double, as aforementioned, is what drives the scaling between the main and secondary coordinate grid. These three are pasted into marked areas in the code, which is then flashed to the brain before each run. The arrays and data are compiled into the brain as constants during runtime.

5.6 Design Decisions & Tradeoffs

Various design decisions and tradeoffs were made in the process of writing this code to ensure optimal performance. The biggest of these involved decisions made with the processing of coordinate data for movement, due to the nature of the data processing pipeline involving a HTML and JS based frontend that was not within the scope of this project.

A flat array for coordinate data encoding from the slicer was chosen over a 2D array or using classes within an array for ease of user interface, as the paste-in data becomes much simpler with a single flat array. Additionally, it was decided as a group that using CNC-style commands was the easiest way to go, as this made the arrays easily modifiable without using the slicer every time, speeding up troubleshooting and creation of test cases on VEX code itself.

In addition, a secondary and main grid dual coordinate system was chosen to decouple the frontend from the physical dimensions of the machine to make the code modular. The fact that all coordinate system scaling happens on the VEX brain during runtime reduces the quantity of data the user has to paste into the program. Moreover, this choice also ensures that all errors related to the scaling stay within VEX code itself rather than the frontend slicer, which was not technically part of the project but more of an add-on, thus keeping all software errors within the scope of this project.

However, this dual-grid coordinate system relies heavily on the cookieDiameter value being accurate. If calibration isn't conducted accurately, every coordinate will be systemically offset. In order to fix this problem and also address the poor drift properties of the motor encoders, another decision was made to conduct periodic re-zeroing every five commands. Doing so removes the reliance on pure encoder accuracy across the entire run and also offsets any potential issues with the initial calibration. However, this induces a pause in the middle of the run which may slow down the overall runtime by a few seconds. Nonetheless, this pause was not of major concern, so the tradeoff was worth it.

An additional design decision was made to hardcode tuning factors for the X and Y gantries individually. In testing, it was discovered that due to the higher load on the X motors, the X motors tended to undershoot a given coordinate while the Y motors overshot. As such, tuning scale factors for movement were incorporated into the code for each gantry axis. However, these tune factors were hardcoded, and were acquired after physical testing rather than analysis, which meant that if the physical configuration changed at all, retuning was required. The greatly reduced complexity of this tuning factor compared to other solutions in solving the individual axis overshoot however did mitigate the cost of having to retune for every change in configuration.

The plunger height tracking per slot was additionally designed in such a way that each syringe slot maintained its own entries for plunger height and barrel position values. When the barrel rotates to a new colour, the current syringe is retracted and the new syringe is advanced only to where it was last left off. This decision was made to avoid re-squeezing icing that's already been dispensed and additionally to keep track of remaining capacity per syringe. The tradeoff to this was relatively minor, coming in the form of increased state management

complexity within the project and the risk of height tracking drifting if a stop/retract sequence is interrupted. Height tracking drifting did become an issue later on, which was solved by adding a forced extrusion of a few mm after each barrel switch to eliminate any drifts.

5.7 Testing

Software testing was conducted primarily through iterative trial runs on the physical machine, with each subsystem tested individually before being integrated into the full sequence. Rather than formal unit testing, the testing approach was largely brute force: it involved repeatedly running the program, observing the output, swapping out parameter values, and re-running until behaviour was acceptable.

The zeroing routine was verified by running it from multiple starting positions and confirming the gantry returned to a consistent home each time. The barrel calibration was tested under the operating lighting conditions by observing the hue values on the VEX home screen and confirming each colour was correctly identified.

Pattern execution was validated by first loading simple known paths such as a horizontal line and a dot. The gantry's physical path across the cookie surface was observed directly against the intended pattern, and motor positions were checked on the brain at key positions (e.g. sudden turns).

The primary focus of the testing was to find the right icing consistency. For each trial, visual observations were made for line continuity, thickness, uniformity, and how defined the dot is. Between each test, motor speeds, thickness of the icing, and timing delays were changed until consistent satisfactory output was achieved.

5.8 Debugging & Final Program

The most persistent issue encountered during development was icing inconsistency. Two distinct failure modes were identified and addressed through iterative debugging.

The first was dripping and smearing caused by the gantry moving too soon after completing a line. When the motors began the next movement immediately after `syringeStop()`, residue icing dangling from the nozzle would drag across the cookie surface rather than falling cleanly. This was resolved by adding deliberate wait delays of 1000–1500 ms after each draw call in `drawLine()` and `drawPoint()`, allowing the icing to settle and any excess to drip clear of the nozzle before the next move began.

The second issue was failure to dispense, where no icing would emerge from the syringe at the start of a line or dot. This was traced to air bubbles trapped in the syringe barrel, which caused the plunger to compress air rather than push icing. To address this, the `syringePreload()` function was introduced, which advances the syringe motor a fixed number of degrees before each draw operation to purge trapped air and bring icing to the nozzle tip. The first draw of a run uses a 30-degree preload to account for a fully unprimed syringe, while subsequent draws use a reduced 20-degree preload since the syringe remains partially primed from the previous operation.

A third issue discovered while testing, which was minor, was that the output patterns

were inverted compared to the user input in the front end. This was a software bug and resulted from the coordinate system being mirrored. While the bulk of software issues were resolved, icing inconsistency remained in the final build as a known, physical limitation, attributable to the inherent variability of icing viscosity and syringe back-pressure across a full dispensing cycle.

6.0 Project Management

6.1 Work Distribution

Work was distributed in accordance with the final project schedule, as seen in Figure 10, to ensure that responsibilities were evenly shared across the team. In the earlier stages of the project, all group members contributed to the initial design and planning including creating the needs statement, functional analysis, engineering design specifications and development of drawings. The project began with Nathan and Trevor primarily focused on creating a 3D model of the assembly while Ryan and Brendan focused on the software component. Towards the end of the project, the entire team collaborated on assembling and testing the machine as well as writing the final report. In an effort to go above and beyond the project requirements, Ryan developed a front-end interface that allows users to create custom designs which are then converted it into arrays that can be interpreted by VEX to control gantry movement and colour selection.

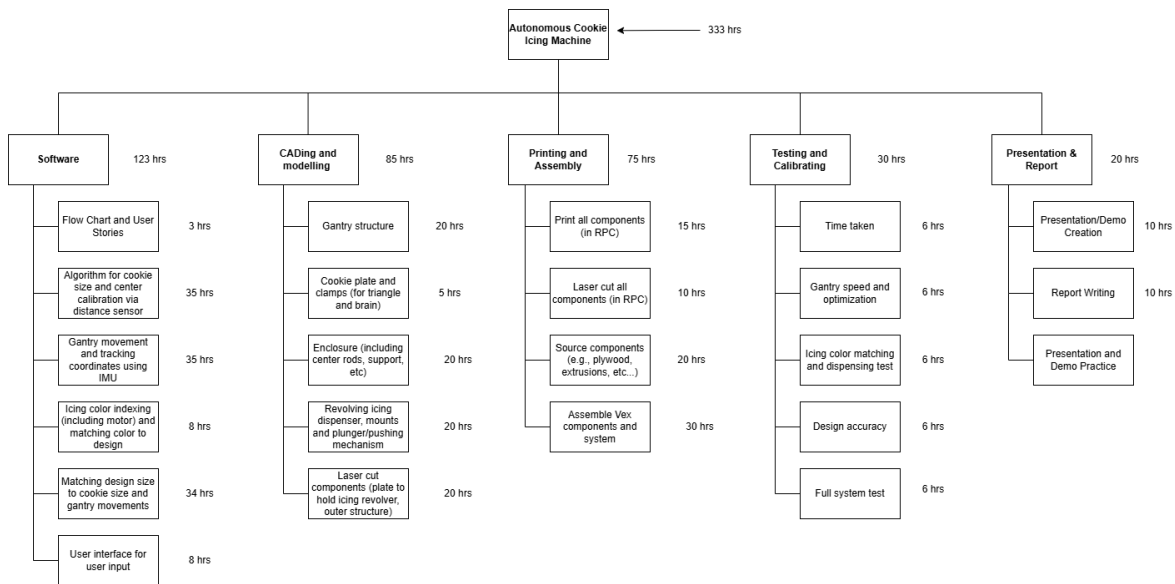


Figure 10: Work Breakdown Structure

6.2 Project Schedules

Figures 11 and 12 below depict the project schedules that were developed organized in a Gantt Chart. The initial schedule, which was made on February 7, underestimated the amount of time required to create the physical prototype which delayed the software development. The dependency of the hardware being constructed before software development was not accounted for which we then added into the final revised schedule. Printing and assembly also took longer than expected due to the need to add more support to the frame to increase stability, which was

unforeseen during the scheduling phase. The presentation and report schedule remained mostly unchanged and on track.

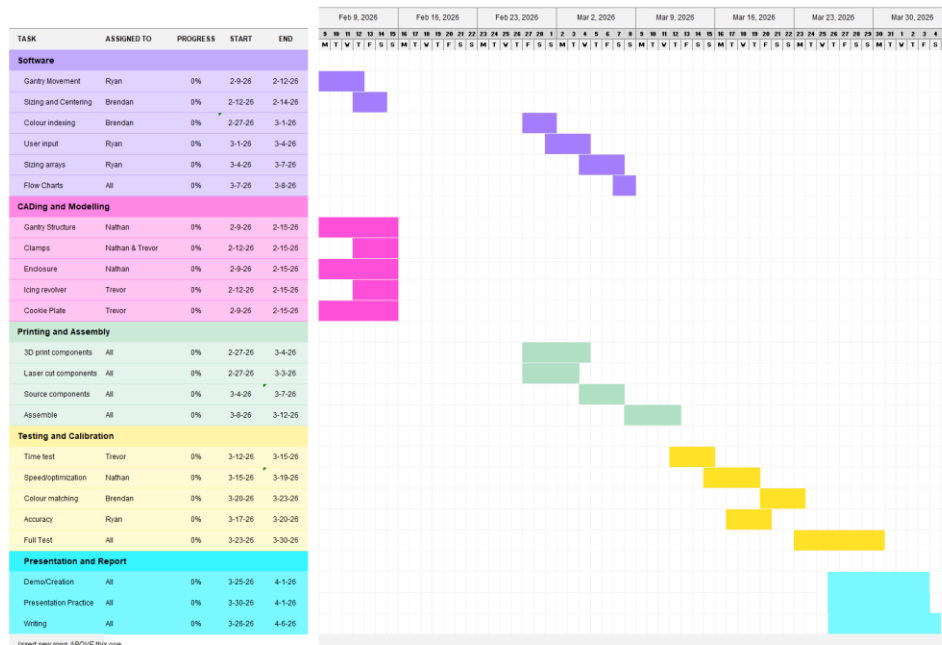


Figure 11: Initial Gantt Chart Schedule

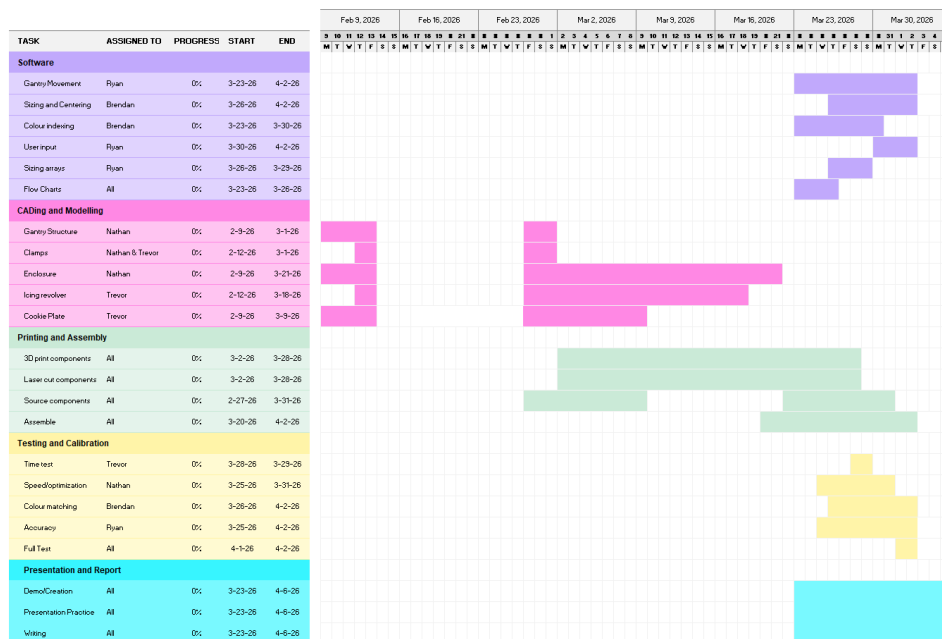


Figure 12: Finalized Gantt Chart Schedule

7.0 Conclusion

7.1 Design Objectives and Features

As a summary, the objective of this project was to design, build and program an automated cookie icing machine using VEX IQ hardware and software that could accurately apply a user-designed icing pattern to a cookie surface, accommodate multiple icing colours, and also successfully complete the entire process in under four minutes. The final design successfully achieved this objective through the use of two integrated systems; a lower two-axis rack and pinion gantry that moved the cookie in the X and Y directions, and an upper icing module with a three-colour icing syringe barrel and motor-driven plunger-based injector system.

A `calibrateCookie` function was written to autonomously detect cookie edges using the distance sensor and derive cookie dimensions at runtime. Due to time constraints, the function could not be tested and validated before the final demonstration, so it was left unused. Instead, the program prompts the user to manually enter the cookie's width and height via the VEX Brain controller before each run. If given more time, integrating `calibrateCookie` into the startup sequence would remove this manual step and make the machine fully autonomous from the moment a cookie is placed. The gantry was constructed from a combination of VEX IQ components alongside custom fabricated acrylic racks, 3D printed structural parts and aluminum guide rods, allowing for 8 inches of travel on each axis independently. The icing module comprised of an acrylic barrel containing the three icing colour syringes, and an acrylic plunger, encased within a larger 3D printed structure. This permitted selection of colours and independent injection of different colours.

On the software side, the program was structured around movement and colour data imported by the user from a custom web-based slicer. The program executed commands sequentially based on two flat arrays containing said data. A zeroing sequence using bumper switches was used to establish a gantry home position for calibration purposes, whilst a `TouchLED` served as the user interface to initiate machine startup. In addition, colour and distance sensors were used to detect parameters for the machine such as icing colour hues available within the machine, and the cookie size.

7.2 Review of Requirements

The final design met most engineering specifications outlined at the start of the project to be considered successful. The machine completed the full icing cycle within the 240-second constraint, successfully applied pre-programmed multi-colour icing patterns to the cookie surface, returned the cookie plate to the zero position after every cycle, and maintained cookie plate stability throughout gantry motion. The three-colour barrel was verified by examination, the print area met the 64 square inch target, and the total cost remained within the \$300 budget. The one specification that was not met was adaptability to different cookie sizes, as the cookies consistently crumbled when placed into the spring plate. Overall, the design successfully demonstrated autonomous cookie icing with repeatable accuracy and met the majority of the engineering requirements.

8.0 Recommendations

8.1 Mechanical Changes

On the mechanical side, stability remained the dominant issue. Hence, the most impactful change would be widening the Y-axis guide rail span. During the testing phase, the rails sat too close to the axis of movement, adding torque which gave the gantry carriage a noticeable wobble that degraded line accuracy. While a stabilizing rod which glided against the bottom surface of the frame was added, it had its own issues of bending and had to be straightened every few minutes.

Additionally, the syringe plunger drive had no force feedback, and the motor would stall against either a fully depressed or overfilled syringe with no way to detect it. Adding a simple spring between the motor shaft and plunger head would prevent stalling and give the system some tolerance for fill variation. Additionally, a bumper at the bottom of plunger travel would also allow proper homing of the Z-axis rather than relying on fixed encoder offsets. This calibration was something that had to be done by hand every time the syringes were refilled.

Another mechanical change that would be made if this project were to be completed again is making the entire machine smaller in general. A lot of material was used to create the frame and a suitable height for the cookie plate, which was not necessarily needed. This increased the total cost and time spent assembling the full machine.

8.2 Software Changes

A recommendation is to add a function to autonomously detect the boundary of the cookie using the distance sensor and derive cookie dimensions at runtime. The program currently prompts the user to manually enter the cookie's width and height via the VEX Brain controller before each run. If given more time, integrating this function into the startup sequence would remove this manual step and make the machine fully autonomous from the moment a cookie is placed.

References

[1] "ChatGPT," OpenAI. [Online]. Accessed: Apr 4, 2026. [Online]. Available: <https://chat.openai.com/>

Appendix A – Full Program Code

```
#pragma region VEXcode Generated Robot Configuration

#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>
#include <math.h>
#include <string.h>

#include "vex.h"

using namespace vex;

brain Brain;

// vex iq macros
#define waitUntil(condition) \
do { \
wait(5, msec); \
} while (!(condition))

#define repeat(iterations) \
for (int iterator = 0; iterator < iterations; iterator++)

inertial BrainInertial = inertial();

void initializeRandomSeed() {
wait(100, msec);
double xAxis = BrainInertial.acceleration(xaxis) * 1000;
double yAxis = BrainInertial.acceleration(yaxis) * 1000;
double zAxis = BrainInertial.acceleration(zaxis) * 1000;
int seed = int(xAxis + yAxis + zAxis);
srand(seed);
}

void vexcodeInit() {
initializeRandomSeed();
}

#pragma endregion VEXcode Generated Robot Configuration

// hardware setup
motor motorXA(PORT12, false); // left x motor
motor motorXB(PORT7, true); // right x motor (reversed)
motor motorY(PORT3, false); // y axis
motor motorBarrel(PORT4, false); // icing barrel rotation
motor motorSyringe(PORT5, false); // syringe plunger (+ = squeeze)
```

```

motor motorPlate(PORT10, false); // cookie plate rotation (calibration)

bumper bumperX(PORT1);
bumper bumperY(PORT2);
touchled startButton(PORT6);
optical barrelOptical(PORT11); // aimed at active syringe
distance distSensor(PORT8); // on nozzle carriage (calibration)

// mechanical constants

const double COUNTS_PER_INCH = 235.3; // needs tuning bad

const double GANTRY_MIN = 0.0;
const double GANTRY_MAX = 8.0;

const double SEC_GRID_RADIUS = 12.0;

const double MAIN_CENTER_X = 4.0;
const double MAIN_CENTER_Y = 4.0;

bool alreadyStarted = true;

// barrel / color constants

const int NUM_COLORS = 3;
const double BARREL_STEP_DEG = 120.0;
const int BARREL_SCAN_SPEED = 5;
const int BARREL_MOVE_SPEED = 15;

// hue windows per slot
// slot 0: blue ~230
// slot 1: red ~352
// slot 2: yellow ~42

const int COLOR_HUE_CENTER[NUM_COLORS] = { 230, 352, 42 };
const int HUE_WINDOW_LO[NUM_COLORS] = { 225, 345, 35 };
const int HUE_WINDOW_HI[NUM_COLORS] = { 235, 360, 45 };
const int HUE_TOLERANCE = 80;

// syringe constants

const int SYRINGE_SPEED_LINE = 3; // finicky, tune carefully
const int SYRINGE_SPEED_DOT = 6;
const int SYRINGE_DOT_MS = 700;
const double SYRINGE_FULL_DEG = 525.0;

// motion constants

const int GANTRY_SPEED = 50;
const int HOME_SPEED = 25;

```

```

const int LINE_SPEED = 40;
const int ZERO_INTERVAL = 5;

const double TUNE_X = 1.0; // x scale factor (probably fine)
const double TUNE_Y = 0.91; // y undershoots to compensate for overshoot

// calibration constants

const int SCAN_SPEED = 15; // slow sweep during prong search
const int PRONG_DETECT_MM = 30; // distance threshold (mm)
const double SEARCH_RANGE = 3.0; // inches either side of centre
const double PRONG_Y_STEP = 0.9; // vertical dist A to BC row — tune this

// ===== PASTE FROM FRONTEND =====

double cookieDiameter = 4.0;

const double DIRECTIONS[] = { 1, -3, 5, -7, -5, 1, 0, -5, -7, -5, 1, 0, -5, -5, 1, 1, 2, 5, 8, 5, 1, 8, 5, 3, -5 };

const int COLORS[] = { 204, 204, 204, 204, 204 };

// ===== PASTE FROM FRONTEND =====

double currentX = 0.0;
double currentY = 0.0;

double barrelColorPos[NUM_COLORS] = { 0.0, 0.0, 0.0 };
int barrelColorHue[NUM_COLORS] = { 0, 0, 0 };
int activeColorIndex = -1;

double plungerHeight[NUM_COLORS] = { 0.0, 0.0, 0.0 };

double syringeDist[NUM_COLORS] = {
    SYRINGE_FULL_DEG,
    SYRINGE_FULL_DEG,
    SYRINGE_FULL_DEG
};

int functionCounter = 0;

double inchesToDegX(double inches) { return inches * COUNTS_PER_INCH * TUNE_X; }
double inchesToDegY(double inches) { return inches * COUNTS_PER_INCH * TUNE_Y; }
double degToInches(double deg) { return deg / COUNTS_PER_INCH; }

double secToMainX(double sx) {
    double scale = (cookieDiameter / 2.0) / SEC_GRID_RADIUS;
    return sx * scale + MAIN_CENTER_X;
}

```

```

double secToMainY(double sy) {
double scale = (cookieDiameter / 2.0) / SEC_GRID_RADIUS;
return sy * scale + MAIN_CENTER_Y;
}

double clampMain(double v) {
if (v < GANTRY_MIN) {
return GANTRY_MIN;
}
if (v > GANTRY_MAX) {
return GANTRY_MAX;
}
return v;
}

void syringeCheck(int slot) {
if (syringeDist[slot] <= 0) {
Brain.Screen.clearScreen();
Brain.Screen.setCursor(1, 1);
Brain.Screen.print("Syringe %d empty!", slot);
wait(1000, msec);
}
}

// =====
// ZEROING
// =====

void zeroGantry() {
Brain.Screen.clearScreen();
Brain.Screen.setCursor(1, 1);
Brain.Screen.print("Zeroing...");

motorXA.setVelocity(HOME_SPEED, percent);
motorXB.setVelocity(HOME_SPEED, percent);
motorY.setVelocity(HOME_SPEED, percent);

motorXA.spin(reverse);
motorXB.spin(reverse);
motorY.spin(reverse);

bool xDone = false;
bool yDone = false;

while (!xDone || !yDone) {
if (!xDone && bumperX.pressing()) {
motorXA.stop(brake);
motorXB.stop(brake);
xDone = true;
Brain.Screen.setCursor(2, 1);
Brain.Screen.print("X zeroed");
}
}

```

```

if (!yDone && bumperY.pressing()) {
  motorY.stop(brake);
  yDone = true;
  Brain.Screen.setCursor(3, 1);
  Brain.Screen.print("Y zeroed");
}
wait(10, msec);
}

motorXA.resetPosition();
motorXB.resetPosition();
motorY.resetPosition();

currentX = 0.0;
currentY = 0.0;

Brain.Screen.clearScreen();
}

// =====
// MOTION: both axes simultaneously
// =====

void moveToMain(double targetX, double targetY, int speed = GANTRY_SPEED) {
  targetX = clampMain(targetX);
  targetY = clampMain(targetY);

  double degX = inchesToDegX(targetX);
  double degY = inchesToDegY(targetY);

  motorXA.setVelocity(speed, percent);
  motorXB.setVelocity(speed, percent);
  motorY.setVelocity(speed, percent);

  motorXA.spinToPosition(degX, degrees, false);
  motorXB.spinToPosition(degX, degrees, false);
  motorY.spinToPosition(degY, degrees, false);

  while (motorXA.isSpinning() || motorXB.isSpinning() || motorY.isSpinning()) {
    wait(10, msec);
  }

  currentX = targetX;
  currentY = targetY;
}

// =====
// MOTION: x axis only
// =====

void moveXOnly(double targetX, int speed = GANTRY_SPEED) {

```

```

targetX = clampMain(targetX);

motorXA.setVelocity(speed, percent);
motorXB.setVelocity(speed, percent);

motorXA.spinToPosition(inchesToDegX(targetX), degrees, false);
motorXB.spinToPosition(inchesToDegX(targetX), degrees, false);

while (motorXA.isSpinning() || motorXB.isSpinning()) {
    wait(10, msec);
}

currentX = targetX;
}

// =====
// MOTION: y axis only
// =====

void moveYOnly(double targetY, int speed = GANTRY_SPEED) {
    targetY = clampMain(targetY);

    motorY.setVelocity(speed, percent);
    motorY.spinToPosition(inchesToDegY(targetY), degrees, false);

    while (motorY.isSpinning()) {
        wait(10, msec);
    }

    currentY = targetY;
}

// =====
// MOTION: secondary grid coords
// =====

void moveToSec(double sx, double sy, int speed = GANTRY_SPEED) {
    moveToMain(secToMainX(sx), secToMainY(sy), speed);
}

// =====
// BARREL: rotate to encoder position
// =====

void rotateBarrelTo(double targetDeg) {
    motorBarrel.setVelocity(BARREL_MOVE_SPEED, percent);
    motorBarrel.spinToPosition(targetDeg, degrees, true);
}

// =====

```

```

// BARREL: nudge barrel encoder back to 0
// =====

void zeroBarrel() {
double pos = motorBarrel.position(degrees);
motorBarrel.setVelocity(BARREL_SCAN_SPEED, percent);
if (pos < 0) {
motorBarrel.spin(forward);
while (motorBarrel.position(degrees) <= 0) {}
motorBarrel.stop(brake);
} else if (pos > 0) {
motorBarrel.spin(reverse);
while (motorBarrel.position(degrees) >= 0) {}
motorBarrel.stop(brake);
}
}

// =====
// BARREL: startup color scan
//
// slowly spins barrel a full 360 (any faster and it freaks out)
// watches optical sensor and records encoder pos for each color
// =====

void indexBarrelColors() {
Brain.Screen.clearScreen();
Brain.Screen.setCursor(1, 1);
Brain.Screen.print("Indexing colors...");

barrelOptical.setLight(ledState::on);
barrelOptical.setLightPower(100);
motorBarrel.resetPosition();

bool hueFound[NUM_COLORS] = { false, false, false };

motorBarrel.setVelocity(BARREL_SCAN_SPEED, percent);
motorBarrel.spin(forward);

while (motorBarrel.position(degrees) <= 360) {
int h = barrelOptical.hue();

for (int i = 0; i < NUM_COLORS; i++) {
if (!hueFound[i] &&
h >= HUE_WINDOW_LO[i] &&
h <= HUE_WINDOW_HI[i]) {

wait(200, msec); // settle

h = barrelOptical.hue();
if (h >= HUE_WINDOW_LO[i] - 10 && h <= HUE_WINDOW_HI[i] + 10) {
barrelColorPos[i] = motorBarrel.position(degrees) - 10.0;
}
}
}
}
}

```

```

barrelColorHue[i] = h;
hueFound[i] = true;

Brain.Screen.clearScreen();
Brain.Screen.setCursor(1, 1);
Brain.Screen.print("Slot %d hue=%d @ %.1f", i, h, barrelColorPos[i]);
wait(500, msec);
}
}
}
}

motorBarrel.stop(brake);

for (int i = 0; i < NUM_COLORS; i++) {
if (!hueFound[i]) {
Brain.Screen.clearScreen();
Brain.Screen.setCursor(1, 1);
Brain.Screen.print("WARNING: slot %d not found", i);
wait(1000, msec);
}
}

zeroBarrel();
// don't pre-rotate or set activeColorIndex here
// leave at -1 so selectColor() moves on first call

Brain.Screen.clearScreen();
Brain.Screen.setCursor(1, 1);
Brain.Screen.print("Color index done");
wait(500, msec);
Brain.Screen.clearScreen();
}

int findColorSlot(int requestedHue) {
int bestSlot = -1;
int bestDiff = 9999;

for (int i = 0; i < NUM_COLORS; i++) {
int diff = abs(barrelColorHue[i] - requestedHue);
if (diff > 180) diff = 360 - diff; // wrap around
if (diff < HUE_TOLERANCE && diff < bestDiff) {
bestDiff = diff;
bestSlot = i;
}
}

Brain.Screen.clearScreen();
Brain.Screen.print("%d", bestSlot);
return bestSlot;
}

```

```

void syringePreload(int angle) {
  motorSyringe.setVelocity(SYRINGE_SPEED_LINE, percent);
  motorSyringe.spinFor(forward, angle, degrees, true);
}

void selectColor(int requestedHue, double (&heights)[NUM_COLORS]) {
  int slot = findColorSlot(requestedHue);
  if (slot == -1) {
    Brain.Screen.clearScreen();
    Brain.Screen.setCursor(1, 1);
    Brain.Screen.print("Color not found! hue=%d", requestedHue);
    wait(1000, msec);
  } else if (slot != activeColorIndex) {

    // save current plunger pos before switching
    if (activeColorIndex >= 0) {
      heights[activeColorIndex] = motorSyringe.position(degrees);
    }

    // retract before rotating barrel
    motorSyringe.spin(reverse, 15, percent);
    while (motorSyringe.position(degrees) >= 0) {}
    motorSyringe.stop(brake);

    rotateBarrelTo(barrelColorPos[slot]);

    activeColorIndex = slot;

    // restore plunger height for new slot
    motorSyringe.setVelocity(SYRINGE_SPEED_LINE, percent);
    motorSyringe.spinToPosition(heights[activeColorIndex], degrees, true);
  }
}

// =====
// SYRINGE
// =====

void syringeStart() {
  motorSyringe.setVelocity(SYRINGE_SPEED_LINE, percent);
  motorSyringe.spin(forward);
  wait(30, msec);
}

void syringeStop() {
  motorSyringe.stop(hold);

  if (activeColorIndex >= 0) {
    double newPos = motorSyringe.position(degrees);
    double dispensed = newPos - plungerHeight[activeColorIndex];
  }
}

```

```

plungerHeight[activeColorIndex] = newPos;
syringeDist[activeColorIndex] -= dispensed;
syringeCheck(activeColorIndex);
}
}

void syringeDot() {
motorSyringe.setVelocity(SYRINGE_SPEED_DOT, percent);
motorSyringe.spin(forward);
wait(SYRINGE_DOT_MS, msec);
motorSyringe.stop(hold);

if (activeColorIndex >= 0) {
double newPos = motorSyringe.position(degrees);
double dispensed = newPos - plungerHeight[activeColorIndex];
plungerHeight[activeColorIndex] = newPos;
syringeDist[activeColorIndex] -= dispensed;
syringeCheck(activeColorIndex);
}
}

void finalRetract(double (&heights)[NUM_COLORS]) {
if (activeColorIndex < 0) {
rotateBarrelTo(barrelColorPos[0]);
activeColorIndex = 0;
}
motorSyringe.setVelocity(50, percent);
motorSyringe.spinToPosition(0.0, degrees, true);
motorSyringe.stop(hold);
}

void incrementAndCheckCounter() {
functionCounter++;
if (functionCounter >= ZERO_INTERVAL) {
functionCounter = 0;
zeroGantry();
}
}

// =====
// DRAW LINE
// =====

void drawLine(double x1, double y1, double x2, double y2, double (&heights)[NUM_COLORS]) {
double mx1 = clampMain(secToMainX(x1));
double my1 = clampMain(secToMainY(y1));
double mx2 = clampMain(secToMainX(x2));
double my2 = clampMain(secToMainY(y2));

wait(1500, msec);

moveToMain(mx1, my1, GANTRY_SPEED);

```

```

if (alreadyStarted) {
  syringePreload(30); // bigger prime on first line
  alreadyStarted = false;
} else {
  syringePreload(20);
}

syringeStart();

motorXA.setVelocity(LINE_SPEED, percent);
motorXB.setVelocity(LINE_SPEED, percent);
motorY.setVelocity(LINE_SPEED, percent);

motorXA.spinToPosition(inchesToDegX(mx2), degrees, false);
motorXB.spinToPosition(inchesToDegX(mx2), degrees, false);
motorY.spinToPosition(inchesToDegY(my2), degrees, false);

while (motorXA.isSpinning() || motorXB.isSpinning() || motorY.isSpinning()) {
  wait(10, msec);
}

currentX = mx2;
currentY = my2;

syringeStop();
incrementAndCheckCounter();
}

// =====
// DRAW POINT
// =====

void drawPoint(double x1, double y1, double (&heights)[NUM_COLORS]) {
  double mx = clampMain(secToMainX(x1));
  double my = clampMain(secToMainY(y1));

  moveToMain(mx, my, GANTRY_SPEED);
  wait(1500, msec);

  if (alreadyStarted) {
    syringePreload(30);
    alreadyStarted = false;
  } else {
    syringePreload(20);
  }

  syringeDot();
  wait(1000, msec);
  incrementAndCheckCounter();
}

```

```

// =====
// EXECUTE PLAN
// =====

void executePlan(const double directions[], int dirSize, const int colors[], int colorSize, double (&heights)[NUM_COLORS])
{
    int dirIdx = 0;
    int cmdIdx = 0;
    bool boundsOk = true;

    while (dirIdx < dirSize && boundsOk) {
        int cmd = (int)directions[dirIdx++];

        if (cmdIdx < colorSize) {
            wait(50, msec);
            selectColor(colors[cmdIdx], heights);
        }

        if (cmd == 1) {
            if (dirIdx + 4 > dirSize) {
                boundsOk = false;
            } else {
                double x1 = directions[dirIdx++];
                double y1 = directions[dirIdx++];
                double x2 = directions[dirIdx++];
                double y2 = directions[dirIdx++];
                drawLine(x1, y1, x2, y2, heights);
            }

        } else if (cmd == 2) {
            if (dirIdx + 2 > dirSize) {
                boundsOk = false;
            } else {
                double x1 = directions[dirIdx++];
                double y1 = directions[dirIdx++];
                drawPoint(x1, y1, heights);
            }

        } else {
            Brain.Screen.clearScreen();
            Brain.Screen.setCursor(1, 1);
            Brain.Screen.print("Cmd ERROR: %d", cmd);
            wait(500, msec);
        }

        cmdIdx++;
    }
}

// calibration; unused

```

```

/*
double sweepXForProng(double startX, double endX) {
moveXOnly(startX, GANTRY_SPEED);

motorXA.setVelocity(SCAN_SPEED, percent);
motorXB.setVelocity(SCAN_SPEED, percent);

motorXA.spinToPosition(inchesToDegX(clampMain(endX)), degrees, false);
motorXB.spinToPosition(inchesToDegX(clampMain(endX)), degrees, false);

while (motorXA.isSpinning()) {
if (distSensor.objectDistance(mm) <= PRONG_DETECT_MM) {
motorXA.stop(brake);
motorXB.stop(brake);
wait(20, msec);
return degToInches(motorXA.position(degrees));
}
wait(5, msec);
}
return -1.0;
}

double sweepYForProng(double startY, double endY) {
moveYOnly(startY, 20);
wait(300, msec);

double targetDeg = inchesToDegY(clampMain(endY));

motorY.setVelocity(5, percent);
motorY.spinToPosition(targetDeg, degrees, false);

wait(100, msec);

while (true) {
int dist = distSensor.objectDistance(mm);
double currentDeg = motorY.position(degrees);

Brain.Screen.setCursor(2, 1);
Brain.Screen.print("Dist: %d mm ", dist);
Brain.Screen.setCursor(3, 1);
Brain.Screen.print("Y: %.2f in ", degToInches(currentDeg));

if (dist > 0 && dist <= 70) {
motorY.stop(brake);
wait(20, msec);
return degToInches(motorY.position(degrees));
}

if (fabs(currentDeg - targetDeg) < 20) {
return -1.0;
}
}
}

```

```

}

wait(50, msec);
}
}

void calibrateCookie() {
Brain.Screen.clearScreen();
Brain.Screen.setCursor(1, 1);
Brain.Screen.print("Calibrating...");

Brain.Screen.setCursor(2, 1);
Brain.Screen.print("Step 1: sweep Y -> A");

moveXOnly(MAIN_CENTER_X, GANTRY_SPEED);
moveYOnly(MAIN_CENTER_Y, GANTRY_SPEED);

double yA = sweepYForProng(GANTRY_MAX, GANTRY_MIN);

if (yA < 0) {
Brain.Screen.clearScreen();
Brain.Screen.setCursor(1, 1);
Brain.Screen.print("ERROR: A not found");
wait(2000, msec);
return;
}

double xA = MAIN_CENTER_X;
Brain.Screen.setCursor(3, 1);
Brain.Screen.print("A: (%.2f, %.2f)", xA, yA);
wait(500, msec);

Brain.Screen.setCursor(2, 1);
Brain.Screen.print("Step 2: sweep X -> B");

double yBC = yA - PRONG_Y_STEP;
moveYOnly(yBC, GANTRY_SPEED);

double xB = sweepXForProng(
MAIN_CENTER_X + SEARCH_RANGE,
MAIN_CENTER_X - SEARCH_RANGE
);

if (xB < 0) {
Brain.Screen.clearScreen();
Brain.Screen.setCursor(1, 1);
Brain.Screen.print("ERROR: B not found");
wait(2000, msec);
return;
}

```

```

Brain.Screen.setCursor(3, 1);
Brain.Screen.print("B: (%.2f, %.2f)", xB, yBC);
wait(500, msec);

Brain.Screen.setCursor(2, 1);
Brain.Screen.print("Step 3: sweep X -> C");

double xC = sweepXForProng(
xB + 0.5,
MAIN_CENTER_X + SEARCH_RANGE
);

if (xC < 0) {
Brain.Screen.clearScreen();
Brain.Screen.setCursor(1, 1);
Brain.Screen.print("ERROR: C not found");
wait(2000, msec);
return;
}

Brain.Screen.setCursor(3, 1);
Brain.Screen.print("C: (%.2f, %.2f)", xC, yBC);
wait(500, msec);

double ax = xA, ay = yA;
double bx = xB, by = yBC;
double cx = xC, cy = yBC;

double D = 2.0 * (ax * (by - cy) + bx * (cy - ay) + cx * (ay - by));

if (fabs(D) < 1e-6) {
Brain.Screen.clearScreen();
Brain.Screen.setCursor(1, 1);
Brain.Screen.print("ERROR: points collinear");
wait(2000, msec);
return;
}

double ux = ((ax*ax + ay*ay) * (by - cy) +
(bx*bx + by*by) * (cy - ay) +
(cx*cx + cy*cy) * (ay - by)) / D;

double uy = ((ax*ax + ay*ay) * (cx - bx) +
(bx*bx + by*by) * (ax - cx) +
(cx*cx + cy*cy) * (bx - ax)) / D;

double R = sqrt((ax - ux)*(ax - ux) + (ay - uy)*(ay - uy));

cookieDiameter = 2.0 * R;
MAIN_CENTER_X = ux;
MAIN_CENTER_Y = uy;

```

```

Brain.Screen.clearScreen();
Brain.Screen.setCursor(1, 1);
Brain.Screen.print("Done!");
Brain.Screen.setCursor(2, 1);
Brain.Screen.print("Diam: %.2f in", cookieDiameter);
Brain.Screen.setCursor(3, 1);
Brain.Screen.print("Ctr: (%.2f,%.2f)", MAIN_CENTER_X, MAIN_CENTER_Y);
Brain.Screen.setCursor(4, 1);
Brain.Screen.print("Press start...");

waitUntil(startButton.pressing());
wait(200, msec);
Brain.Screen.clearScreen();
}
*/

// =====
// PROGRAM MAIN
// =====

int main() {
  vexcodeInit();

  Brain.playNote(4, 0, 150);
  wait(30, msec);
  Brain.playNote(4, 2, 150);
  wait(30, msec);
  Brain.playNote(4, 4, 150);
  wait(30, msec);
  Brain.playNote(5, 0, 400);

  Brain.Screen.clearScreen();
  Brain.Screen.setCursor(1, 1);
  Brain.Screen.print("Press start...");
  waitUntil(startButton.pressing());
  wait(200, msec);
  Brain.Screen.clearScreen();

  int dirSize = sizeof(DIRECTIONS) / sizeof(DIRECTIONS[0]);
  int colorSize = sizeof(COLORS) / sizeof(COLORS[0]);

  zeroGantry();

  //calibrateCookie(); // finds diameter + center, writes globals
  moveToSec(0.0, 0.0);
  wait(200, msec); // visual inspection pause
  indexBarrelColors();
  executePlan(DIRECTIONS, dirSize, COLORS, colorSize, plungerHeight);
  moveToSec(0.0, 0.0);
  zeroGantry();

```

```
finalRetract(plungerHeight);
```

```
Brain.Screen.clearScreen();  
Brain.Screen.setCursor(1, 1);  
Brain.Screen.print("Done!");
```

```
Brain.playNote(4, 0, 150);  
wait(30, msec);  
Brain.playNote(4, 2, 150);  
wait(30, msec);  
Brain.playNote(4, 4, 150);  
wait(30, msec);  
Brain.playNote(5, 0, 400);
```

```
return EXIT_SUCCESS;  
}
```

